

Multilevel Security (MLS)

1. What is Multi-Level Security (MLS)?

Multi-Level Security, or MLS, is a way of protecting information by dividing it into different security levels. Each level shows how sensitive the information is. If unauthorized people access it, the higher the level, the greater the damage.

MLS is mostly used in military, government, and other high-security organizations. It controls who can access information based on the person's security clearance.

Key Features

- **Different protection for each level:** Every level of data has its own rules for storage, sharing, and access.
- **Compartmentalization:** Information is divided into separate sections called compartments. A person may access one compartment but not another, even at the same security level.
- **Contextual security:** The system applies rules dynamically, based on the situation. This is called dynamic policy enforcement.
- **Strong assurance:** MLS controls can be checked and verified independently by security assessors.

Case Study: Military Intelligence System

A military department stores classified information in one system. Information is divided into four levels: Unclassified, Confidential, Secret, and Top Secret. Each officer gets a clearance level that matches their role.

- A junior officer with Confidential clearance can only open confidential documents.
- A senior officer with Top Secret clearance can open highly sensitive reports.
- A civilian employee may only see unclassified information.

If a user tries to open information above their clearance, the system denies access automatically.

Compartmentalization Example

Suppose two projects, Project Eagle and Project Falcon, are both Top Secret. An officer working on Project Eagle still cannot see Project Falcon files unless separately authorized. This stops unnecessary exposure of sensitive data.

Dynamic Policy Example

If a user tries to access Top Secret files from an unknown device or outside the secure network, the system may block access, ask for extra authentication, or alert administrators.

2. Why is Multi-Level Security (MLS) Important?

MLS helps organizations protect sensitive information and control access more effectively.

Key Benefits

- **Classification and labelling:** Data is marked as Confidential, Secret, or Top Secret, so users know how to handle it.
- **Access control:** Access can be limited by user, device, location, or clearance level, lowering the risk of unauthorized access.

- **Dynamic policy enforcement:** Rules apply automatically depending on the situation, such as denying access from an unsafe network.
- **Contextual security:** Decisions are based on labels, tags, roles, and other attributes.
- **Granular access control:** Detailed restrictions can be applied within or between security domains, so users see only what they need.

Exam Tip: MLS improves data protection, lowers security risk, and gives strong control over sensitive information.

3. Difference between ABAC and MLS

ABAC and MLS are both access control approaches, but they work in different ways.

Aspect	ABAC	MLS
Definition	Access is granted based on attributes of the user, resource, and environment.	Access is decided using security clearance levels and classifications.
Model Type	Policy-based and flexible.	Hierarchical and rigid.
Control	Fine-grained control using rules and conditions.	Coarse-grained control using fixed levels.

4. ABAC and MLS Security Policies

ABAC Policy Example

A doctor can access patient records only during working hours, and only if assigned to the same department. This decision uses attributes such as user role, user department, resource type, and the time of access.

MLS Policy Example

A user with Top Secret clearance can access Top Secret, Secret, Confidential, and Unclassified documents. A user with only Confidential clearance cannot open Secret or Top Secret files.

Integration of MLS and ABAC

Combining ABAC with MLS makes a security system more powerful and flexible. MLS controls access using security levels, while ABAC adds another layer by checking attributes and conditions in real time.

Attributes are pieces of information about the user, device, location, or environment, usually written as key-value pairs, for example Nationality = Australian, or Location = Canberra. Access is granted only when all required conditions are met. Even a user with correct clearance may be denied if they connect from an unauthorized country or unsafe device.

Advantages of Combining MLS and ABAC

- **Faster and more accurate access:** The system quickly decides who should see specific information in different situations.
- **Better compartment management:** Access is controlled more precisely using conditions such as location, device, or project.
- **Network simplification:** Fewer separate systems are needed, making operations easier to manage.
- **Secure information sharing:** Supports safe sharing between countries or organizations in joint missions.

- **Faster secure collaboration:** Users can create and share files quickly while the system still enforces the owner's security rules.

5. Handling Users and Data in MLS

MLS controls access using two things: the classification of the data, and the clearance level of the user. Common classification levels are Top Secret, Secret, Confidential, and Unclassified. The system compares the user's clearance with the data's classification before granting access.

No Read Up

A user cannot read information above their clearance level. Example: a user with Confidential clearance cannot open Secret or Top Secret files. This rule stops unauthorized disclosure.

No Write Down

A user cannot move or copy information from a higher security level to a lower one. Example: Top Secret data cannot be saved into a Confidential system. This rule prevents classified information from leaking into less secure places.

Compartmentalization

MLS may also restrict access by project, department, or need-to-know basis. Even users with the same clearance level may not see the same information unless they are authorized for that specific compartment. For example, two Top Secret officers on different projects cannot see each other's files without extra permission.

Importance of These Rules

Together, these rules make sure users only access information they are officially allowed to handle. This reduces:

- Unauthorized access
- Accidental data leaks
- Insider threats
- Intentional security breaches

6. Bell–LaPadula Model

The Bell–LaPadula Model is a security model built to protect confidentiality. It is mainly used in MLS environments such as military and government systems. Both users (subjects) and data (objects) are assigned security levels.

Common Security Levels

1. Top Secret — highly sensitive military or national security information.
2. Secret — sensitive information such as banking or financial data.
3. Confidential — business transactions and internal company information.
4. Public — information available to everyone, such as product lists.

Main Rules of Bell–LaPadula

- **Simple Security Property (No Read Up):** A user cannot read data above their clearance. Example: a Confidential user cannot read Secret files. This stops unauthorized access.
- **Star (*) Property (No Write Down):** A user cannot send data to a lower security level. Example: Top Secret data cannot be saved as Confidential. This prevents leaks.
- **Strong Star Property (No Write Up or Down):** A user can write only at their own level, giving even stronger protection.

Exam Tip: Bell–LaPadula protects Confidentiality — remember it by "No Read Up, No Write Down."

7. Biba Model

The Biba Integrity Model was created by Kenneth J. Biba in 1975. Unlike Bell–LaPadula, which protects confidentiality, Biba protects integrity — keeping data correct, trustworthy, and free from unauthorized change.

Main Rules of Biba

- **Simple Integrity Property (No Read Down):** A user cannot read data from a lower integrity level. Example: a trusted manager should not read untrusted data, so low-quality information cannot affect them.
- **Star (*) Integrity Property (No Write Up):** A user cannot write data to a higher integrity level. Example: a junior employee cannot change trusted financial records, which stops data corruption.

Goals of the Biba Model

5. Protect data from unauthorized modification.
6. Prevent authorized users from making improper changes.
7. Keep data consistent, both internally and externally.

8. Differences between Bell-LaPadula and Biba Model

Aspect	Bell-LaPadula Model	Biba Model
Primary Goal	Data Confidentiality — stop unauthorized disclosure.	Data Integrity — stop unauthorized modification.
Main Use Case	Military, defense, classified environments.	Financial systems, healthcare, industrial control systems.
No Read Rule	No Read Up — cannot read higher classification.	No Read Down — cannot read lower integrity data.

Aspect	Bell-LaPadula Model	Biba Model
No Write Rule	No Write Down — cannot write to lower classification.	No Write Up — cannot write to higher integrity level.
Trust Model	Lower-level users are not trusted with higher secrets.	Lower-level data is not trusted to influence higher processes.
Access Focus	Who can see the data.	Who can change the data.

9. Inference in Database Security

Inference is a security risk where unauthorized users work out confidential information from normal, allowed queries, even without direct access to the sensitive data. Attackers study query results, metadata, statistics, or access patterns to guess restricted information.

How an Inference Attack Works

Inference attacks get around normal access controls by collecting and combining non-sensitive data to reveal confidential facts.

Example of an Inference Attack

A hospital database blocks direct access to disease records, but an attacker runs indirect queries. Query 1 asks how many patients have cancer, and gets 100. Query 2 asks how many male patients aged 30 to 35 have cancer, and gets 1. The attacker can now work out that the only man in that age group has cancer, which breaks patient confidentiality.

Types of Inference Attacks

Type	Description	Example
Statistical Inference	Uses statistical queries to extract private information.	Querying average salaries to guess an individual salary.
Data Correlation	Combines data from several sources to reveal hidden details.	Cross-referencing voting records with public data to identify voters.
Metadata Inference	Studies metadata such as logs and access patterns.	Monitoring access logs to identify VIP accounts.
Aggregation Inference	Uses SUM, COUNT, or AVG to infer individual data.	Using total sales to guess a competitor's revenue.

Countermeasures

Defense	Description	Example
Query Restriction	Blocks queries that return small, unique results.	Requiring a minimum number of records per query.
Noise Addition	Adds random variation to prevent exact inference.	Slightly changing values in statistical reports.
Data Masking	Hides sensitive attributes in results.	Showing partial credit card numbers only.
Differential Privacy	Ensures results do not reveal any one individual.	Used by Apple and Google for user analytics.
Cell Suppression	Hides specific cells to prevent exposure.	Removing unique salary values from small reports.
Role-Based Access	Limits data exposure based on user role.	Medical staff see only data relevant to their department.

Real-World Applications

- **Healthcare (HIPAA):** Protects patient data from statistical inference.

- **Finance (PCI-DSS):** Prevents inference of salary and transaction data.
- **Government and Census (GDPR):** Uses data anonymization to protect identities.
- **Cloud and AI Systems:** Protects data from adversarial attacks and unauthorized learning.

10. Architectures of Database Integrity

Database integrity means data stays accurate and consistent throughout its life. To achieve this, database architecture is divided into two layers: Physical Integrity and Logical Integrity.

Physical Integrity

Physical Integrity protects the storage layer. It keeps data intact even during hardware failure, power loss, natural disasters, or corruption.

- **Redundancy and hardware:** RAID, ECC memory, and UPS protect against hardware failure.
- **Software and logging:** Backup systems, transaction logs, and journaling rebuild data after a failure.

Logical Integrity

Logical Integrity makes sure data is accurate, meaningful, and follows the rules of the database schema and business logic. The Four Pillars of Logical Integrity are:

- **Entity Integrity:** Every table has a primary key, and no part of it is NULL, so each record is unique.
- **Referential Integrity:** Foreign keys correctly point to primary keys in related tables.
- **Domain Integrity:** Data values must fall within an acceptable range or format.
- **User-defined Constraints:** Custom business rules, such as salary always being greater than zero.

Enforcing Logical Integrity

- **SQL Constraints:** NOT NULL, UNIQUE, CHECK, and FOREIGN KEY.
- **Programmable Logic:** Triggers and stored procedures for complex validation.
- **Application-level Validation:** Checks performed before data reaches the database.

Example: A system must stop a student enrolment record from pointing to a course ID that does not exist. This is a classic case of referential integrity.

Physical vs Logical Integrity

Feature	Physical Integrity	Logical Integrity
Focus	Storage reliability and error recovery.	Data validity and logical consistency.
Concerned with	Hardware or software failures.	Rule violations and relational consistency.
Protection Methods	Backup, journaling, RAID, fault tolerance.	Constraints, triggers, schema design.
Example Threat	Disk failure corrupts data.	Entering a negative value for age.
Layer	System or hardware level.	Database schema and application level.

11. Different Techniques: Checksum and Hash Function

What is a Checksum?

A checksum is a small, usually numeric value computed from a block of data, used to detect accidental corruption. It is simple, fast, and suitable for checking physical integrity.

Example: A stored file has a checksum saved alongside it. When retrieved, a new checksum is computed and compared with the old one. If they differ, the data was altered or corrupted.

- **In databases:** Storage engines such as MySQL InnoDB use checksums to detect corrupted pages.
- **Transaction logs:** Checksums validate log consistency.
- **Backup tools:** Checksums confirm that backup and replicated data matches.
- **Limitation:** Simple checksums are not cryptographically secure and can miss intentional tampering.

What is a Hash Function?

A hash function converts input data of any size into a short, fixed-length value called a hash, hash code, or message digest. If even one letter of the input changes, the output hash changes completely.

Key Characteristics of a Cryptographic Hash Function

- **Deterministic:** The same input always produces the same hash.
- **Fast to compute:** Hashes are generated quickly, even for large files.
- **One-way (irreversible):** It is practically impossible to recover the original data from its hash.
- **Collision resistant:** It is extremely hard to find two different inputs with the same hash.

Common Hash Functions

- SHA-256 — very secure and widely used.
- SHA-3 — a newer, highly secure function.
- BLAKE2 — fast and secure, used in modern applications.
- MD5 — older, no longer considered secure.

Example: Checking File Integrity with a Hash

When a document is first saved, the system calculates its SHA-256 hash and stores it. Later, when the document is opened, a new hash is calculated and compared to the stored one. If the values match, the document is unchanged. If they differ, the document has been modified or corrupted.

Checksums vs Hash Functions

Feature	Checksums	Hash Functions
Purpose	Detect accidental corruption.	Detect tampering and verify integrity.
Security Level	Low (not cryptographic).	High (if cryptographic hash is used).
Speed	Very fast.	Fast, but slower than checksums.
Usage Example	Disk block verification.	Password hashing, digital signatures.
Common Algorithms	CRC32, Adler-32, XOR.	SHA-256, SHA-3, MD5 (not recommended).

12. Digital Signature

A digital signature is an electronic method that proves a digital message is genuine, unchanged, and sent by the correct person. It is like a handwritten signature, but far more secure because it uses cryptography.

A digital signature answers three questions: Who sent the message (authenticity)? Has it been changed (integrity)? Can the sender deny sending it (non-repudiation)?

How a Digital Signature Works

A digital signature uses two keys: a Private Key kept secret by the sender to create the signature, and a Public Key shared with everyone to verify it.

Steps of a Digital Signature

8. The sender writes a message.
9. A hash function creates a unique hash of the message.
10. The sender encrypts this hash with their private key. This is the digital signature.
11. The sender sends the message and the signature together.
12. The receiver decrypts the signature with the sender's public key to get the original hash.
13. The receiver also calculates a new hash from the received message.
14. If both hash values match, the signature is valid.

Example

Arif wants to send Bobby a message about transferring money. The computer calculates a hash of the message, and Arif encrypts this hash with his private key to create the digital signature. He sends the message and the signature to Bobby. Bobby decrypts the signature using Arif's public key, and also calculates the hash of the message herself. Since both hash values match, Bobby confirms the message is really from Arif and has not changed.

Advantages of a Digital Signature

- **Authenticity:** Only the sender owns the private key, so only they can create a valid signature.
- **Integrity:** Changing even one letter of the message changes its hash, so tampering is detected.
- **Non-repudiation:** Since the signature needs the sender's private key, the sender cannot deny sending it.

Real-Life Uses

- Secure emails — verifying the sender and that the content is unchanged.
- Software downloads — checking that a program was not modified by attackers.
- Electronic contracts — both parties verify the signed agreement.
- Online banking — securing financial transactions.
- Blockchain and cryptocurrencies — proving a transaction was authorized by the wallet owner.
- Government services — online tax filing, e-certificates, and passports.

13. Practical Strategies for Maintaining Integrity

Several practical strategies help organizations maintain the integrity of systems and databases.

Enforce Integrity Constraints

Database constraints are one of the most effective integrity tools. A primary key gives every record a unique identifier, and a foreign key maintains relationships between tables. Unique constraints stop duplicate values, and NOT NULL constraints make sure important fields are never empty. Check constraints restrict allowed values, such as requiring age to be zero or greater.

Apply Input Validation and Sanitization

Input validation checks that data is entered in the correct format and within acceptable limits before it is stored. Input sanitization removes or blocks harmful characters that could be used in attacks such as SQL injection. Common techniques include whitelisting allowed values, using regular expressions, and escaping special characters.

Use Transactions and ACID Properties

Related database operations should run as a transaction, following the ACID principles: Atomicity, Consistency, Isolation, and Durability. These ensure that either all operations succeed or none do. For example, in a bank transfer, both the withdrawal and deposit must succeed together, or the whole transaction is cancelled.

Maintain Secure Access Control

Only authorized users should view or change data. Each user should get only the permissions needed for their job, following the principle of least privilege. Organizations often use Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC) to manage who can read, update, or delete data.

Implement Logging and Auditing

Every important action, such as an insert, update, or delete, should be recorded in a log or audit trail. If a problem happens, administrators can review logs to see who made changes, when, and what data was affected.

Use Cryptographic Methods

Hash functions such as SHA-256 create a unique fingerprint of data, so any change is easy to detect. Digital signatures add extra protection by verifying both data integrity and the identity of the creator.

Enable Version Control and Backups

Keeping earlier versions of records allows recovery from accidental changes or errors. Regular backups should be created, stored securely, and tested to confirm that data can be restored after failures or attacks.

Deploy Integrity Monitoring Tools

Tools such as Tripwire and AIDE compare the current state of files with previously stored values. If unauthorized changes are found, the tool alerts administrators immediately.

Automate Consistency Checks

Automated scripts or monitoring tools can find duplicate records, missing table relationships, invalid values, and orphan records, which reference data that no longer exists. This improves data quality and reduces manual effort.

Train Users and Enforce Policies

Technology alone cannot guarantee integrity. Regular training helps employees learn correct data entry and recognize mistakes. Clear data governance policies keep data accurate, consistent, reliable, and compliant.

14. Tools and Policies for Enforcement

Secure environments use a mix of technical tools and administrative policies to protect data and ensure compliance with security standards.

Access Control Mechanisms

- **RBAC:** Grants permissions based on user roles.
- **ABAC:** Uses attributes such as location, device, and clearance level for dynamic decisions.
- **MAC:** Enforces strict rules based on system-defined classifications, as used in MLS.

Database Security Tools

- **Database Activity Monitoring (DAM):** Tracks and audits database activity in real time.
- **Encryption:** Secures data at rest and in transit using algorithms such as AES and TLS.
- **Data Masking and Redaction:** Hides sensitive data from unauthorized users.
- **Integrity Constraints:** Rules such as foreign keys, unique keys, and NOT NULL conditions.

System-Level Enforcement Tools

- **SIEM:** Collects, analyzes, and reports on security events across systems.
- **EDR:** Monitors and responds to threats at the device level.
- **HIDS:** Monitors file integrity, system logs, and configuration changes.

Cryptographic Techniques

- **Checksums and Hashing:** Validate data integrity, for example using SHA-256.
- **Digital Signatures:** Ensure data authenticity and integrity.
- **PKI:** Manages digital certificates for secure authentication and encryption.

Administrative Policies and Standards

- **Security Policies:** Define roles, responsibilities, and procedures for secure operations.
- **Data Classification Policy:** Specifies how data is labelled and protected by sensitivity.
- **Audit and Logging Policy:** Requires regular review and archiving of logs.
- **Change Management Procedures:** Ensure system changes are controlled and approved.